Static Analysis And CI/CD Pipelines

Martin Joo

Introduction

In this short essay, I'd like to talk about two important things:

- CI/CD with Github actions and Gitlab pipelines
- Static analysis

They're two very different topics but running static analysis is one of the responsibilities of a CI/CD pipeline, so they're also related. With Github, I'll show you an example that requires no previous knowledge of DevOps. However, the Gitlab example is much more complicated, which uses docker and docker-compose, so you need to know these technologies to understand the pipeline.

Static Analysis

In general, a static analysis tool helps you avoid:

- Bugs
- Too complex methods and classes
- Lack of type-hints
- Poorly formatted code

There are an infinite amount of tools out there, but in the following pages, I'd like to show my three favorite tools.

phpinsights

This is my number #1 favorite tool. It gives you an output like this:

It scores your codebase on a scale from 1..100 in four different categories:

- Code. It checks the general quality of your code. It doesn't involve any style of format, but only quality checks such as:
  - Unused private properties
  - Unnecessary final modifiers
  - Unused variables
  - Correct switch statement
  - And much more
- Complexity. In my opinion, **this is the most critical metric**. It simply checks how complicated a class is, using cyclomatic complexity. It's a fancy way of saying: how many different execution paths exist in a function. Or put it simply: how many if-else or switch statements do you have in one function or class. By default, it raises an issue if the average is over 5.
- Architecture. It checks some general architectural stuff, such as:
  - Method per class limit
  - Property per class limit
  - Superfluous interface or abstract class naming
  - Function length
  - And so on
- Style. It checks some style-related rules, for example:
  - No closing PHP tag at the end of files
  - There's a new line at the end of files
  - Space after cast

By default, it doesn't require any configuration at all. Of course, if you don't want to use some rules, you can disable them. Check out the documentation for more information. phpinsights can be run by issuing this command:

```
./vendor/bin/phpinsights analyse
```

It gives you an excellent summary and an interactive terminal where you can see every issue. But it also ships with a 'non-interactive' mode, and you can also define the minimum scores you want to have:

```
./vendor/bin/phpinsights --no-interaction --min-quality=80 --min-
complexity=90 --min-architecture=70 --min-style=75
```

The `--no-interaction` flag means that the terminal window does not expect any input; it just gives you the summary and every error message. The other `--min-xy` flags make it possible to define the minimum scores for each category. For example, if the complexity score drops below 90%, the command will yield a non-zero output and an error message. **The minimum complexity score is always 90% for me.**

This is good for us because it can be automated (see later).

larastan

Larastan is a Laravel specific tool built on top of phpstan. These two use the same configuration format and rule system. It ships with a default ruleset (very strict) and has a config parameter called `level`. This parameter determines how strict it is and how many rules are applied. If you want to learn more about these rules, check the documentation.

The config file looks like this:

```
includes:
  - ./vendor/nunomaduro/larastan/extension.neon

parameters:
  paths:
    - app
    - src

  level: 5
  ignoreErrors:
    - '#PHPDoc tag @var#'
  excludePaths:
    - 'app/Http/Kernel.php'
    - 'app/Console/Kernel.php'
    - 'app/Exceptions/Handler.php'
  checkMissingIterableValueType: false
  noUnnecessaryCollectionCallExcept: ['pluck']
```

Important values are:

- We need to use src to scan all of the domains in the `paths`.
- `level` goes from 1..9. Basically, you need to experiment with what level is best for you, but here's my general rule:
  - Legacy project: start with 1. In my opinion, you have no other options if the static analysis is new to the project.
  - Fresh application: somewhere between 4 and 6, but it heavily depends on the team and the project.
  - Never reach for level 9. Seriously, it gets pretty hard above level 5. My all-time best was level 7, and I was dying during the process. It's like a tough game where you cannot beat the final boss.
- Under the `excludePaths` you can list files or directories that you want to exclude. Sometimes I exclude default Laravel files such as the ones above.

You can browse the full config in the `phpstan.neon` file (root directory of the sample app).

You can run the rules with this command:

```
./vendor/bin/phpstan analyse
```

deptrac

This tool helps you to clean up your architecture. In the book, I used several classes. The important ones are:

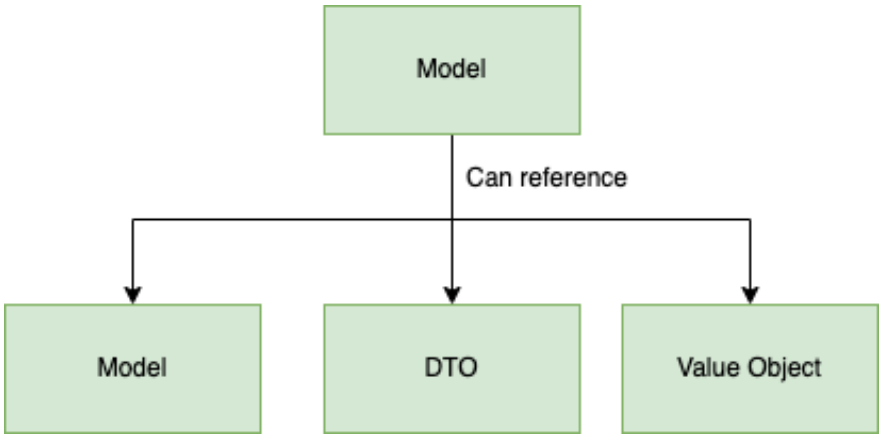
- Controllers
- Action
- ViewModels
- Builders
- Models
- DTOs

Each class has its purpose. For example, I don't want a controller to start implementing business logic. It has three responsibilities, in my opinion:

- Accepting a request.
- Calling the necessary methods from another class.
- Returning a response.

Another important rule is to keep the models lightweight. If the models are sending notifications, dispatching jobs, or calling APIs, it's a bad design, in my opinion.

If you think about it, these architectural rules can be enforced by simply defining which class a Model can reference, right? Something like that:



deptrac does precisely that. If these rules are not followed, and a Model uses a Job deptrac will throw a huge red screen in your face.

We can configure these rules in the `deptrac.yaml` file:

```
parameters:
  paths:
    - ./app
    - ./src
  exclude_files:
    - '#.*test.*#'
    - '#.*Factory\.php$#'
  layers:
    - name: Action
      collectors:
        - type: className
          regex: .*Actions\\.*
```

This tells deptrac that the project has a layer called Action, and the files can be collected using this regex `.*Actions\\.*` It means that every file inside the `Actions` folder is an action class. After the layers are created, we can define the rulesets:

```
ruleset:
    Controller:
      - Action
      - ViewModel
      - Model
      - DTO
      - ValueObject
    Action:
      - Event
      - Model
      - DTO
      - Builder
      - ValueObject
    Model:
      - Builder
      - Model
      - DTO
      - ValueObject
    DTO:
      - Model
```

```
      - DTO
      - ValueObject
  ValueObject:
      - ValueObject
```

This means that a model in the project can only use:

- Other models.
- Query builders.
- DTOs.
- Value objects.

If anything else is referenced inside a model, it will throw an error. You can find the config in the `deptrac.yaml` inside the sample application. If you want to run it, just run this command:

```
./vendor/bin/deptrac analyse
```

So these are my favorite static analysis tools. But the true power comes when you integrate these tools into your CI/CD pipeline.
I highly recommend using this tool in new projects. It requires only 15-30 minutes to set up, but it provides value for the next 3-5 years. Also, if you have a legacy project that you want to clean up, this package can be really helpful!

CI/CD

Github

Now let's see how these tools can be automated in a CI/CD pipeline using Github Actions. The first step is to create a `.github/workflows/laravel.yml` file in your project. The filename can be anything. This step can be done using the Github UI.

Now let's see the contents of the file:

```
name: Laravel

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

Once again, the name can be anything that describes this particular action. However, the `on` is quite important. It tells Github when to run the action. This example will run in two scenarios:

- When you push to the `main` branch.
- When someone creates a pull request to the `main` branch.

Next, we have jobs. Each job does "something" such as:

- Running tests.
- Running static analysis.
- Deploying your code to a server.

```
jobs:
  quality-check:
    runs-on: ubuntu-latest
```

In this example, I have only one job called `quality-check`. It will run the tests and static analysis. As you can see, it runs on Ubuntu. So when you push a commit to the main branch, Github will run this action on one of their servers and snip up an Ubuntu container for you. These servers are called runners. By default, Github gives you some runners you can use, but you can also use your own servers.

Each job has some steps. Each step is a command that you want to run. Each job runs on a fresh Ubuntu instance, so these steps will include stuff like:

- Checking out the code
- Installing composer dependencies

before running the tests or static analysis. The great thing about Github actions is that we can use some predefined steps, for example:

```
steps:
  - uses: shivammathur/setup-php@15c43e89cdef867065b0213be354c2841860869e
    with:
      php-version: '8.1'
  - uses: actions/checkout@v2
```

The first step will set up PHP 8.1, and the second one will check out your code to the runner server. These steps are so common that Github provides them for us. Or, to be more precise, the first step comes `shivammathur`. So this step is an open-source repo.

After that, we need to run some typical commands just as if we were installing a new Laravel project on our local machines:

```
- name: Copy .env
  run: php -r "file_exists('.env') || copy('.env.example', '.env');"
- name: Install Dependencies
  run: composer install -q --no-ansi --no-interaction --no-scripts --no-
progress --prefer-dist
- name: Generate key
  run: php artisan key:generate
- name: Directory Permissions
  run: chmod -R 777 storage bootstrap/cache
```

These are the same steps you would do by hand:

- Creating an `.env` file from the `.env.eample`
- Installing composer dependencies
- Generating a key
- Making the storage folder 777

When running CI/CD pipelines, we often use sqlite databases because they are blazing fast. They're not the best option for every project, but for this demo, it's perfect (later, I'll show you how to run MySQL and Redis in docker-compose). To use sqlite we need to create a database file:

```
- name: Create Database
  run: |
    mkdir -p database
    touch database/database.sqlite
```

It makes a `database` directory and a `database.sqlite`. The `|` operator is how you can run multiple commands in the same step.

As the next step, we can run phpinsights, larastan, and deptrac:

```
- name: Run phpinsights
  run: ./vendor/bin/phpinsights --no-interaction --min-quality=80 --min-complexity=90 --min-architecture=70 --min-style=75
- name: Run phpstan
  run: ./vendor/bin/phpstan analyse --memory-limit=1G
- name: Run deptrac
  run: ./vendor/bin/deptrac analyse
```

And finally, we can run the tests:

```
- name: Test
  env:
    DB_CONNECTION: sqlite
    DB_DATABASE: database/database.sqlite
  run: php artisan test
```

With the `env` you can set environment variables. And basically, that's it! Now, we have an action that:

- Installs the project from scratch.
- Runs three static analysis tools
  - phpinsights
  - larastan
  - deptrac
- Sets up a sqlite DB
- Runs the tests

Using Github UI, you can get a lot of these steps from templates! The whole workflow looks like this:

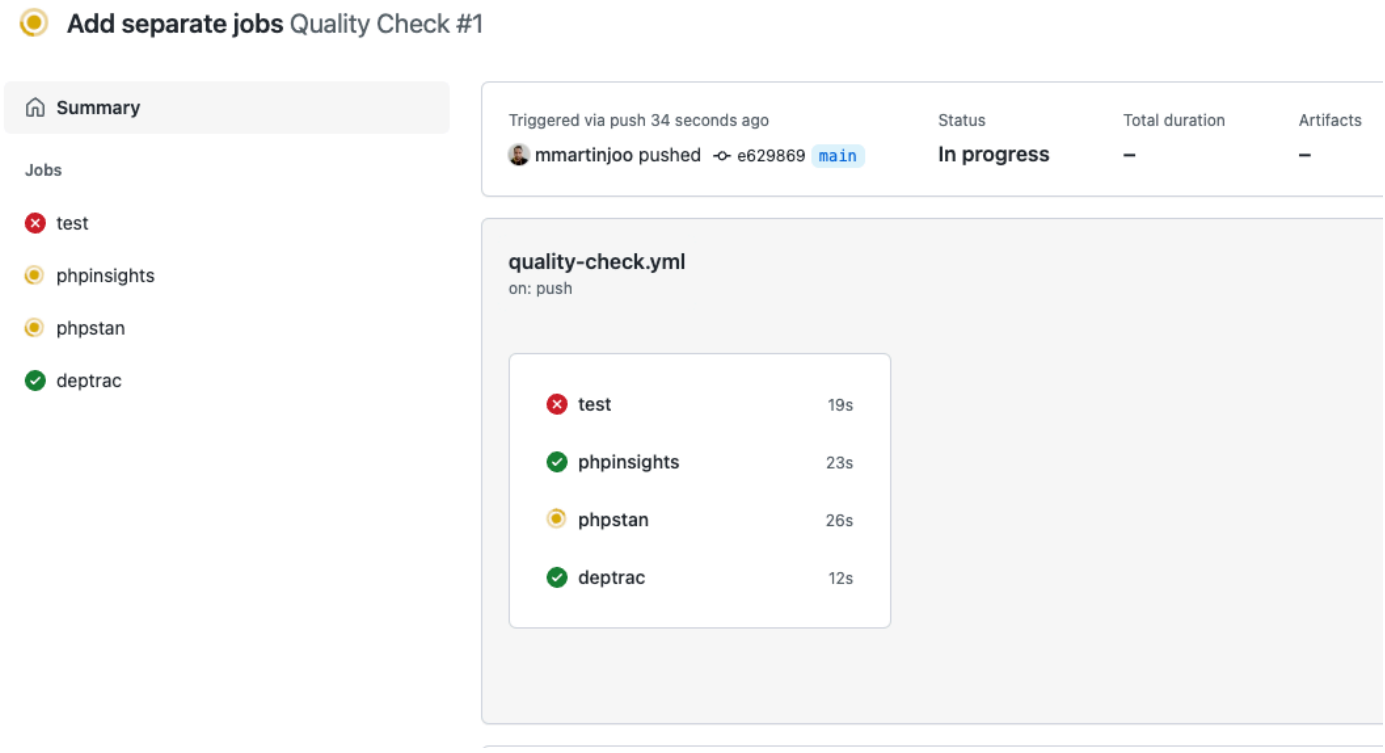
```
name: Quality Check
```

```yaml
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  quality-check:

    runs-on: ubuntu-latest

    steps:
      - uses: shivammathur/setup-php@15c43e89cdef867065b0213be354c2841860869e
        with:
          php-version: '8.1'
      - uses: actions/checkout@v2
      - name: Copy .env
        run: php -r "file_exists('.env') || copy('.env.example', '.env');"
      - name: Install Dependencies
        run: composer install -q --no-ansi --no-interaction --no-scripts --
no-progress --prefer-dist
      - name: Generate key
        run: php artisan key:generate
      - name: Directory Permissions
        run: chmod -R 777 storage bootstrap/cache
      - name: Create Database
        run: |
          mkdir -p database
          touch database/database.sqlite
      - name: Run phpinsights
        run: ./vendor/bin/phpinsights --no-interaction --min-quality=80 --
min-complexity=90 --min-architecture=70 --min-style=75
      - name: Run phpstan
        run: ./vendor/bin/phpstan analyse --memory-limit=1G
      - name: Run deptrac
        run: ./vendor/bin/deptrac analyse
```

```
      - name: Test
        env:
          DB_CONNECTION: sqlite
          DB_DATABASE: database/database.sqlite
        run: php artisan test
```

You can check out the quality-check.yml in the downloaded files. This is a great setup; it has only one disadvantage: every step runs after the previous one. But if you think about it, these steps do not depend on each other, right? So we can run phpinsights and phpstan parallel. We have two options to do that:

- Create a separate workflow for each step. This means we have a dedicated yml file for phpinsights, phpstan, deptrac, and testing. These workflows will run parallel, and the whole pipeline will fail if one fails. This is exactly what we want. We only have to move some code from this file to other files. I included this solution inside the "Separate Workflows" folder.
- The other option is to create different jobs inside one workflow. This is also very easy to do; I included an example inside the "Separate Jobs" folder. If you choose this option, jobs will run parallel, and you have a nice output in Github:



If you have a deploy job after tests and static analysis, you can use the `needs` key to define the order between jobs:

```yaml
jobs:
  test: ...
  phpinsights: ...
  phpstan: ...
  deptrac: ...
  deploy:
    needs: [test, phpinsights, phpstan, deptrac]
```

In this example, the deploy job will run only when the other jobs are finished successfully.

Gitlab

Now, let's see an example with Gitlab. It's going to be more complicated than the previous one. It also uses docker, so you should check out the basic usage if you're not familiar with it.

In Gitlab, we have stages:

```
stages:
    - build
    - analyze
    - test
    - publish
    - deploy
```

You can think about them as containers for the actual jobs. You can also set some globally accessible variables:

```
variables:
    JOB_IMAGE_NAME_DEV: $CI_REGISTRY_IMAGE:dev
    JOB_IMAGE_NAME_PROD: $CI_REGISTRY_IMAGE/prod:$CI_COMMIT_SHORT_SHA
```

The variables that start with `$CI_` are provided by Gitlab. For example, the `$CI_COMMIT_SHORT_SHA` contains the first six characters of the commit SHA. In this example, I'm using docker and docker-compose, so the first step is to build a docker image:

```
build-dev:
    stage: build
    script:
        - docker build --target=dev -t $JOB_IMAGE_NAME_DEV  -f
./docker/Dockerfile .
    tags:
        - do-runner
```

In this example, I build a docker image and give a tag something like this:

```
martinjoo/my-repository:dev
```

This is the value of the `$JOB_IMAGE_NAME_DEV` variable. I also build a production image:

```
build-prod:
    stage: build
    script:
        - docker build --target=prod -t $JOB_IMAGE_NAME_PROD -f
./docker/Dockerfile .
    tags:
        - do-runner
```

You can see that this job targets the `prod` stage of the Dockerfile. Some typical differences between dev and prod images:

- dev image contains xdebug to create a test coverage report.
- It contains all of the `require-dev` dependencies from `composer.json`.
- It ships with a dev `php.ini` that enables xdebug and `error_reporting` is set to `E_ALL`.

Meanwhile, a prod image:

- Does not contain xdebug.
- Does not contain dev dependencies, only production ones.
- It ships with a production `php.ini` with `display_errors` set to `off`, `memory_limit` set to a value that makes sense.

After we have the images, we can run static analysis:

```
phpstan:
    stage: analyze
    script:
        - docker run --rm -t $JOB_IMAGE_NAME_DEV ./vendor/bin/phpstan
analyze --memory-limit=1G
    tags:
        - do-runner

phpinsights:
    stage: analyze
    script:
        - docker run --rm -t $JOB_IMAGE_NAME_DEV php artisan insights --no-
interaction --min-quality=85 --min-complexity=88 --min-architecture=88 --min-
style=86
    tags:
        - do-runner
```

```
deptrac:
    stage: analyze
    script:
        - docker run --rm -t $JOB_IMAGE_NAME_DEV ./vendor/bin/deptrac
    tags:
        - do-runner
```

As you can see, the command is the same, but everything runs in the container this time. After everything passes, we can run the tests:

```
test:
    stage: test
    before_script:
        - docker-compose -f docker-compose.ci.yml up -d --force-recreate
        - docker-compose -f docker-compose.ci.yml exec -T demo-project chown
-R www-data:www-data /usr/local/src
    script:
        - docker-compose -f docker-compose.ci.yml exec -T demo-project php
artisan test
    after_script:
        - docker cp demo-project_1:/usr/local/src/demo-
project/tests/coverage tests/coverage
        - docker-compose -f docker-compose.ci.yml down
    tags:
        - do-runner
    artifacts:
        when: always
        paths:
            - tests/coverage
```

Running tests requires the full stack to run, meaning:

- API (PHP app with Laravel)
- MySQL
- Redis

So this time I need to start a docker-compose with every service. The compose file uses the `$JOB_IMAGE_NAME_DEV` variable as the image name, so the same docker image will be used. In this example, `demo-project` is the name of your service in the docker-compose.

After the tests have passed, we can push the image to a registry:

```
push-dev:
    stage: publish
    script:
        - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD
$CI_REGISTRY
        - docker push $JOB_IMAGE_NAME_DEV
    tags:
        - do-runner
```

In this case, I'm using Gitlab's registry to log in with the environment variables provided by Gitlab.

To see the complete examples check out the following files in the sample application:

- `.github/workflows/laravel.yml`
- `.gitlab-ci.yml`